

AI辅助高层管理决策

目标

- 利用AI帮助高层管理人员更方便地获取所需信息
- 在平台中嵌入生成式BI（商业智能）或报告界面

功能需求

功能点	说明
智能问答	高层管理人员可以通过自然语言查询获取答案
可视化展示	支持生成图表和表格，帮助直观理解数据
嵌入平台	AI功能直接集成在现有平台中，方便使用

重点：AI不仅提供文字答案，还能生成图表和表格，提升决策效率。

后续要求与步骤

后续说明

- 在推进项目之前，需要先完成其他若干要求
- 具体要求将在后续详细告知

备注

- 当前方向为优先实现AI辅助查询和报告功能
- 后续将根据需求调整和完善功能设计



项目启动与客户意向

初步试点计划

- 已有几个感兴趣的客户，适合开展初步试点
- 目前基础设施尚未确定，需要进一步明确方向
- 希望通过试点后，能够明确后续发展路线

会议洞见

- 如果决定使用 **AWS Bedrock**，建议管理员与 AWS 联系人沟通
- 争取获得 AWS 的支持和资源协助

“希望通过这次试点，能够更清晰地确定未来的基础设施和发展方向。”



项目开发策略

主要目标

- 避免从零开始构建定制化的大型语言模型（LLM）
- 以加快开发速度为核心，快速推出最小可行产品（MVP）
- 通过MVP向客户展示，获取反馈后继续迭代改进

开发流程

阶段	说明
技术选型	采用推荐的技术栈，避免自定义复杂开发
准备阶段	快速完成准备工作，缩短项目启动时间
MVP开发	迅速开发出最小可行产品，便于展示和测试
客户反馈	向客户展示MVP，收集反馈意见
后续迭代	根据反馈调整和优化产品，推动项目进展

重点：快速迭代和客户反馈是推动项目成功的关键。

AI能力预期

文档结构说明

- 文档从高层次描述AI的预期功能开始
- 明确AI系统应达到的目标和能力范围

预期功能

- 设定AI可实现的核心功能和性能指标
- 作为后续开发和评估的参考标准



AI 销售演示准备

演示目标

关键点

- 通过销售演示让潜在客户产生“顿悟”时刻（aha moment）
- 让客户对AI的能力产生明确预期
- 通过展示准备好的提示（prompts）来实现以上目标

演示内容设计

复杂示例

- 总结预算
- 发现预算中的漏洞（leakage）
- AI自动计算金额及漏洞数值

风险与注意事项

- 复杂问题可能导致AI产生“幻觉”（hallucination）
- 需确保信息来源准确，避免虚假内容

演示形式建议

演示形式	说明	价值
表格展示	以表格形式呈现数据和结果	直观清晰，便于理解
可视化图表	通过图表展示数据关系	增强视觉冲击力，提升说服力
文件生成	允许客户生成所需文件	增强互动性和实用性

重点：确保演示内容真实可靠，避免AI生成错误信息，提升客户信任感。



AI在复杂与简单问题中的应用

复杂与简单问题的分类

复杂问题

- 例如：销售推介中的复杂问题
- 需要AI进行深入分析和回答

简单问题

- 例如：直接查询采购订单（PO）的总金额
- 采购订单由询价单（RFQ）转换而来
- 适合快速、直接的答案

AI应用的基础与挑战

数据与分析基础

- 基于已有的分析信息进行回答
- 依赖现有数据资源

现有分析的局限性

局限性	说明
使用率低	用户对分析工具的使用不充分
理解不足	用户对分析结果理解不全面
数据非实时	数据更新滞后，影响准确性

未来改进方向

- 从现有分析经验中学习
- 提供更优的AI解决方案
- 实现更实时、更易用的数据支持

05:14



会议讨论内容总结

讨论主题一：产品内部化的重要性

细节

- **内部化 (internalization)**：强调不希望将不利产品信息暴露给外部。
- 目标是确保产品相关内容**仅限内部使用**，避免外泄。
- 这是团队普遍认可的做法，认为这是“好主意”。

讨论主题二：Bedrock平台及其定制化能力

细节

- Bedrock支持****代理(agent)****功能。
- 需要明确Bedrock上代理的**定制化程度**。
- 团队希望构建类似**LangGraph**的系统，具备以下特点：

特点	说明
企业级	适合企业使用，具备高可靠性和安全性
可控性	用户可以对系统进行全面控制
定制化	支持根据需求进行灵活调整

- 目标不仅是企业级别，更重要的是**构建可控且高度定制化的系统**。

“我们想做的是类似LangGraph的东西，不仅仅是企业级，更是能够控制和定制的系统。”

06:07

LLM提供商与代理架构

主要内容

- **Bedrock** 作为大型语言模型（LLM）的提供商
- **LangChain** 作为代理(agent)的运行平台
- 可能采用的**混合架构**
- Bedrock 是否支持**代理工作流编排(agentic workflow orchestration)**
- 架构设计和组件的开放性问题

细节说明

主题	说明
Bedrock角色	作为LLM的提供者
LangChain角色	代理(agent)的宿主平台
混合架构	Bedrock提供模型，LangChain运行代理，形成混合使用方案
代理工作流编排	Bedrock可能支持，但具体使用范围和程度尚未确定
架构和组件问题	设计细节和组件选择仍是开放问题，需管理员进一步确认
产品定位	首个产品将面向内部使用，作为初步试点

“Bedrock也有代理工作流编排，如果我没记错的话，但具体能用到什么程度还是个开放问题，管理委员会是最合适的答复者。”

产品开发与决策

产品方向

- 首个产品定位为**内部使用**
- 该决策被团队成员认可，认为是合理且可行的第一步

讨论重点

- 确认架构设计的合理性
- 明确Bedrock与LangChain的协同方式
- 后续由管理员解答架构相关的具体问题

“我完全支持这个决定，首个产品做内部使用是个好选择。”

06:50

系统架构设计

高层架构设计概述

- 目前讨论的是一个**高层次的架构设计**，尚未最终确定。
- 架构分为**用户端代理 (user-facing agent)** 和**后端代理 (backend agents)**，不同代理负责不同任务。
- 设计目标是确保用户端代理能够**可靠地理解**和**转换**用户输入。

代理分类与功能

- 根据当前的分析，初步设想了**三种可能的代理**。
- 需要进一步在数据库中核实这些代理的可行性和具体功能。

代理类型	主要职责	备注
用户端代理	负责与用户交互，理解用户需求	需适应非技术用户的输入习惯
后端代理1	处理特定任务或数据分析	待确认具体功能
后端代理2	处理特定任务或数据分析	待确认具体功能

用户交互与挑战

用户端代理的关键要求

- 用户端代理必须能够**准确转换用户的输入**，保证信息传递的准确性。
- 假设用户多为**非技术背景**，可能不会像熟悉聊天机器人的用户那样有效地进行提示 (prompting)。

重要提示：

许多用户在构建领域中技术水平有限，导致他们的输入提示可能不够精准，用户端代理需要对此有较强的适应能力。

08:46

输出质量与提示优化

影响输出质量的因素

- 提示训练
 - 可能存在提示 (prompt) 设计不佳的问题
 - 通过训练用户或系统提升提示质量，改善输出结果
- 输出质量波动
 - 提示质量直接影响最终输出的质量

“提示设计和训练是影响输出质量的重要因素。”



数据时效性与数据库架构问题

数据时效性问题

- 数据延迟
 - 分析数据存在24小时延迟
 - 数据每天仅更新一次，非实时数据
- 数据使用困境
 - 是否继续使用已有的延迟数据存在疑问
 - 延迟数据可能导致分析结果不准确或滞后

数据库架构与解决方案

- 当前架构缺陷
 - 数据库架构未能支持实时数据更新
- 改进方向
 - 需要技术团队寻找更优的数据使用方案
 - 探索更合理的数据库架构以提升数据时效性

问题点	现状描述	可能影响	改进建议
数据时效性	24小时延迟，每日更新	数据非实时，影响准确性	寻找实时或更频繁更新方案
数据库架构	不支持实时更新	限制数据使用效率	优化架构，支持实时数据

“技术团队需提供更优方案，解决数据时效性和架构问题。”



快速网站数据更新问题

现状描述

详细说明

- 当前快速网站（Quick Site）用于生成公共用户界面
- 数据更新机制：数据仅在午夜时分推送更新
- 结果：当天新增的数据无法即时在快速网站上显示
- 举例说明：当天上午创建的数据，需等到第二天早上才能在快速网站上看到

设计改进建议

详细说明

- 现有的快速网站数据更新限制需取消
- 目标：设计一个能够处理**实时数据**的系统
- 可能性：通过设计AI系统，实现实时数据更新和展示

重要提示： 当前快速网站的数据更新延迟影响用户体验，设计实时数据处理系统是解决方案。

10:18



实时数据获取

实时数据需求

- 不需要等待一天后才能查看数据
- 可以获取**实时数据**，无延迟
- 使用自有的**MCP服务器**来获取实时数据，需要自行搭建

Quick Site的限制

- 使用Quick Site时存在**24小时延迟**
 - 若不使用Quick Site，则可实现实时数据访问
-

多租户安全性

多租户平台需求

- 平台支持**多租户**（multiple tenant）
- 不同租户之间的数据必须**隔离**
- 不同实体不能访问其他实体的数据

现有功能

- 该功能已在分析系统和平台中实现
- 需要确保该功能同样应用于**AI系统**

重点：确保多租户数据隔离功能在所有系统中一致且有效。

11:34

数据保密与访问权限管理

数据保密的重要性

细节

- **数据保密**是指限制不同用户对数据的访问权限，确保敏感信息不被未授权人员查看。
- 当前系统中，**不同实体的数据不可被其他实体查看**，保证了数据的隔离性和安全性。

用户访问权限的限制与管理

细节

- 未来考虑限制用户只能访问特定的AI代理，以控制数据访问范围。
- 例如：

用户角色	访问权限描述	访问范围示例
实体经理	可访问所有AI代理，查看整个实体的所有数据	查看整个平台的采购订单（PO）数据
采购员	访问权限受限，不能访问所有AI代理	无法查询全公司本月采购订单总价值

重要观点：通过角色区分和权限限制，确保不同用户只能访问其职责范围内的数据，提升数据安全性和管理效率。

12:03

AI访问控制与权限管理

AI访问限制机制

访问限制原因

- 不允许某些AI访问特定AI以生成报告
- 通过限制访问，防止未经授权的数据生成行为

访问替代方案

- 用户可以请求其他AI执行不同任务
- 多功能AI系统支持多样化操作

实体与用户数据权限管理

实体数据访问限制

- 无法将所有实体数据集中到单一实体中
- 数据“提升”（lift）操作受限，防止数据过度集中

用户级别访问控制

- 每个实体包含多个用户
- 针对不同用户设置不同的数据访问权限
- 通过权限管理控制用户从AI获取数据的能力

技术实现

- 自建MCP服务器作为中间层
- AI调用已有API接口，确保数据访问安全与合规

关键点	说明
访问限制	不允许某些AI生成报告
替代方案	可请求其他AI执行不同任务
实体数据管理	无法集中所有实体数据
用户权限	针对用户设置访问控制
技术手段	自建MCP服务器调用现有API接口

总结： 通过多层次的权限控制和技术手段，确保AI系统中数据访问的安全性和合规性，防止未经授权的数据生成和访问。

AI调用API的认证与授权

认证与授权需求

细节

- 为了让AI调用现有API，必须对用户进行认证和授权
- 认证授权流程类似于当前前端到后端的流程
- 这是一个必要的前提条件，但具体实现需要设计基础设施
- 设计基础设施的任务需要依赖相关人员完成

重点：认证和授权是AI调用API的基础保障，必须先解决

应用集成与界面设计

应用集成现状

细节

- 目前尚无固定的UI设计方案
- 可能采用右下角弹出窗口的形式展示聊天界面
- 本质上是一个聊天机器人（chatbot）
- 目前仅限于管理员（executives）访问该页面

界面设计思路

设计要点	说明
访问入口	管理后台页面
展示形式	类似其他聊天机器人，弹出式聊天窗口
用户范围	仅限执行人员访问
未来扩展	可根据需求在其他位置增加入口

总结：初期界面设计以简洁实用为主，后续可根据需求调整和扩展

14:24



会议内容概要

讨论主题

主要内容

- 当前讨论的是聊天功能的初步粗略编辑版本
- 未来会根据技术团队的架构和需求进行更精确的修改
- 目标是实现用户与AI的互动

关键点

事项	说明
当前状态	粗略编辑版本，尚未最终确定
未来计划	根据技术团队的具体架构和产品需求调整
用户互动方式	多种方式均可，用户与AI的互动不是问题

细节说明

用户与AI互动

- 目前只是一个初步的想法
- 用户可以通过多种方式与AI进行互动
- 互动方式灵活，不存在技术障碍

重要提示： 用户与AI的互动方式可以多样化，当前阶段不必担心具体实现细节。



成本计算与团队协作

成本计算需求

细节

- 业务团队需要进行**成本计算**
- 成本计算是下一次会议的**必要准备内容**
- 需结合**技术角度**和**业务角度**共同讨论
- 讨论内容包括选择使用的**工具**

进展与预期

细节

- 成本会根据所选的****大型语言模型（LLM）****有所不同
- 目前已有一些示例，但需要在**开发阶段**进一步确认
- 预计在**一个月内**能获得更清晰的成本数据
- 成本信息将在**工作阶段**而非生产后阶段得知

“我们将在开发阶段就了解成本情况，而不是等到生产后才知道。”



估算与成本分析

估算基础

估算方法

- 初步估算可以基于单个单位（如一个请求或一次调用）进行
- 估算结果会随着版本更新有所变化
- 需要关注版本变化对成本和时间线的影响

成本构成

LLM（大型语言模型）成本

- LLM成本是主要成本，但不是唯一成本
- 使用LangGraph时，相关工具是开源的，无额外费用
- 使用AWS Bedrock时，需支付Bedrock平台的费用，主要针对LLM部分
- 具体使用哪个LLM模型，会在开发过程中决定

成本估算示例

成本项	说明	备注
LLM成本	主要成本，依赖所选模型	如GPT-4o, Sonnet等
平台费用	如AWS Bedrock的使用费用	仅针对平台服务
工具费用	如LangGraph等开源工具无费用	开源免费

重要提示：选择合适的LLM模型（如GPT-4o）即可满足需求，无需使用更高版本（如GPT-5）。



设计与时间规划

设计依据

- 依据现有文档进行架构设计
- 文档内容是否足够支持设计、成本和时间规划需确认

时间线规划

- 时间线需结合成本估算和架构设计同步制定
- 版本变化可能影响时间线，需要动态调整

以上为文本内容的结构化笔记，涵盖了估算方法、成本构成及设计与时间规划的核心要点。

18:10



大型语言模型（LLM）成本估算与架构设计

成本估算

估算原则

- 可以基于某个基线大型语言模型（LLM）进行成本估算。
- 估算带有很大的不确定性和免责声明，成本可能上下波动。
- 但有观点认为成本只会上升，不会下降。

重要观点：成本“永远不会下降，只会上升”。

架构设计

代理部署方案

- 计划采用**A2A**协议，在Bhuta平台上**单独部署各个代理（agent）**。
- 每个模块（例如采购模块）是否作为独立代理，需要进一步确认。
- 目前观察到不同模块的提问意图存在重叠。

讨论点

议题	说明
代理部署方式	使用A2A协议，代理独立部署
平台	Bhuta
模块是否为代理	例如采购模块是否作为独立代理待定
意图重叠问题	不同模块提问意图存在交叉

18:57



会议讨论与后续安排

会议内容概述

- 每个代理（agent）将只部署在AWS的ECS上。
- 目前采用该方案作为暂时的解决方案。
- 计划进行深入的架构设计，设计完成后再进行讨论。

后续步骤与时间安排

事项	说明
架构设计	由负责人进行深度架构设计
下一次会议时间	设计完成后进行更新和进一步讨论
预计时间	下周三或下周四
参与人员	Anand及相关团队成员

“我们可以在架构准备好之后再开会，给Anand一些时间。”

19:45



项目进展与会议安排

项目进展跟踪

细节

- 正在进行相关工作，会持续更新进展
- 预计下周完成后，可以进行对接

会议安排

细节

- 计划下周召开会议
- 会议内容包括：

议题	说明
架构分享	由Edmund介绍整体架构
费用估算	讨论项目的预算
时间线规划	确定项目整体时间安排
项目启动	明确项目启动时间和步骤

- 会议时间初步定在周一或周二，待确认具体日期
- 会后将在群聊中跟进会议纪要，确保所有成员了解讨论内容及后续任务

重要提醒： 需要在群聊中总结会议内容，方便所有人知晓讨论结果和后续工作安排。

20:36



会议结束与沟通总结

会议结束确认

- 会议主持人多次确认是否有问题
- 参与者均表示无其他问题
- 会议气氛轻松，交流顺畅

后续沟通安排

发送文档链接

- 会议结束时提到将文档链接发送到群组
- 方便无法访问文档的成员提出问题
- 提供联系人以便随时沟通

重要提醒

“如果你无法访问文档，可以联系我。”



会议礼仪与表达

感谢与告别

- 多次表达感谢
- 礼貌结束会议
- 使用“谢谢”、“再见”等礼貌用语

语言特点

- 口语化表达频繁出现，如“lah”、“bro”等
- 体现轻松、友好的沟通氛围